

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №4

з дисципліни “**Веб-технології та програмування (ч.2)**”

на тему “**Створення найпростішого HTTP-сервера на Node.js.**”

Виконав:

Студент 2 курсу

групи ФєП-23

Пилипчук Любомир

Перевірив:

Асистент кафедри системного

проектування

Гусак Олег Васильович

Мета роботи

- Використати навички програмування на JavaScript та роботи з форматами JSON/XML
- Запрограмувати асинхронні виклики для читання файлу
- Навчитися обробляти запити та надсилати відповідь з допомогою вбудованого модуля HTTP

Теоретичний матеріал

- Лекція 4 - Node.js модулі та структура проекту. Практика програмування з Node.js
- Лекція 5 - Практика програмування з Node.js
- Лекція 6 - Поняття про асинхронне програмування
- Лекція 5 - Практика програмування з Node.js
- Лекція 7 - Протокол HTTP. Простий сервер на Node.js

Підготовка до роботи

- Створіть новий репозиторій з назвою backend-course-2025-4
- Встановіть ім'я користувача у форматі Ваше Ім'я bc2025-4 та вашу електронну пошту для коміту з допомогою команди git config
- Закомітте у ваш репозиторій json файл з даними відповідно до вашого варіанту
 - Варіант 1 - bank_managers.json - Інформація про керівників банків, визначені в статті 42 Закону України “Про банки і банківську діяльність”.

Хід роботи

Частина 1 - спільні параметри командного рядка

- Ваша програма (незалежно від варіанту) має приймати наступні аргументи:
 - -i, --input - (обов'язковий параметр) шлях до файлу, який даємо для читання
 - -h, --host - (обов'язковий параметр) адреса сервера

- -p, --port - (обов'язковий параметр) порт сервера
- Програма має виводити помилку, якщо файл для читання не знайдено. Текст помилки: "Cannot find input file".
- Програма має виводити помилку, якщо не задано обов'язковий параметр.
- З допомогою модуля http запустіть веб сервер. Значення параметрів --host, --port мають бути передані у метод http.Server.listen()
- Для того, щоб сервер автоматично перезапускався кожного разу, коли ви змінюєте

програму, запустіть сервер з допомогою пакета nodemon, цей пакет встановіть як

залежність лише для розробки.

- Виконайте вимоги цієї частини та закомітьте результат.

Частина 2 - читання файлу та обробка запиту

- Використовуйте асинхронні виклики readFile та writeFile для читання та запису

файлів (модуль fs Node.js) у цій частині.

- Програма має виконувати завдання незалежно від того, які значення мають поля прочитаного з файлу JSON. Тобто якщо значення полів будуть інші, програма все одно має коректно виконувати свою задачу.
- Створіть програму яка запускає веб-сервер і для кожного запиту:
 - читає JSON з файлу
 - аналізує його вміст відповідно до завдання
 - формує XML і надсилає його у відповідь на запит до вашого серверу
- Для формування XML використайте пакет fast-xml-parser. Встановіть його, додайте до залежностей package.json. Дослідіть документацію і у звіті окресліть основні особливості роботи з пакетом.
- Виконайте завдання відповідно до варіанту нижче та закомітьте

Варіант 1: bank_managers.json

Параметри URL query для фільтрації (аналог параметрів -m, -mfo та -n, -normal):

- ?mfo=true: Відображати код МФО банку перед назвою (поле MFO).
- ?normal=true: Відображати лише працюючі банки з кодом 1 “Нормальний” (фільтрація за полем COD_STATE).

Приклад виводу інформації у форматі XML (якщо задано обидва параметри, вихідні поля:

MFO, NAME, COD_STATE):

```
<banks>
<bank>
<mfo_code>305880</mfo_code>
<name>АТ "КБ "ЗЕМЕЛЬНИЙ КАПІТАЛ"</name>
<state_code>1</state_code>
</bank>
<bank>
<mfo_code>380731</mfo_code>
<name>АТ "Дойче Банк ДБУ"</name>
<state_code>1</state_code>
</bank>
</banks>
```

```
const { Command } = require('commander');
const http = require('http');
const fs = require('fs');
const fsPromises = require('fs').promises;
const { XMLBuilder } = require('fast-xml-parser');

const program = new Command();

program
  .helpOption('-e, --help')
  .option('-i, --input <path>')
  .option('-h, --host <address>')
  .option('-p, --port <number>');

program.parse(process.argv);
const options = program.opts();

if (!options.input) {
```

```

    console.error("Cannot find input file");
    process.exit(1);
}

if (!options.host) {
    console.error("Please, specify the server host");
    process.exit(1);
}

if (!options.port) {
    console.error("Please, specify the server port");
    process.exit(1);
}

if (!fs.existsSync(options.input)) {
    console.error("Cannot find input file");
    process.exit(1);
}

const builder = new XMLBuilder({
    ignoreAttributes: false,
    format: true,
});

const server = http.createServer(async (req, res) => {
    try {
        const rawData = await fsPromises.readFile(options.input, 'utf-8');
        const banks = JSON.parse(rawData);

        const reqUrl = new URL(req.url, `http://${options.host}:${options.port}`);
        const isMfoRequested = reqUrl.searchParams.get('mfo') === 'true';
        const isNormalRequested = reqUrl.searchParams.get('normal') === 'true';

        const resultItems = [];

        banks.forEach(bank => {
            if (isNormalRequested && bank.COD_STATE !== 1) {
                return;
            }

            let bankData = {};
            if (isMfoRequested && bank.MFO) {
                bankData.MFO = bank.MFO;
            }

            bankData.NAME = bank.N_K || bank.NAME || bank.SHORTNAME || "Unknown
name";
            bankData.COD_STATE = bank.COD_STATE;

            resultItems.push(bankData);
        });

        const xmlObj = {

```

```

        root: {
            bank: resultItems
        }
    };

    const xmlContent = builder.build(xmlObj);

    res.writeHead(200, { 'Content-Type': 'application/xml; charset=utf-8' });
    res.end(xmlContent);

} catch (error) {
    console.error("Error processing request:", error.message);
    res.writeHead(500, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end("Internal Server Error");
}
});

server.listen(options.port, options.host, () => {
    console.log(`Server is listening on http://${options.host}:${options.port}`);
});

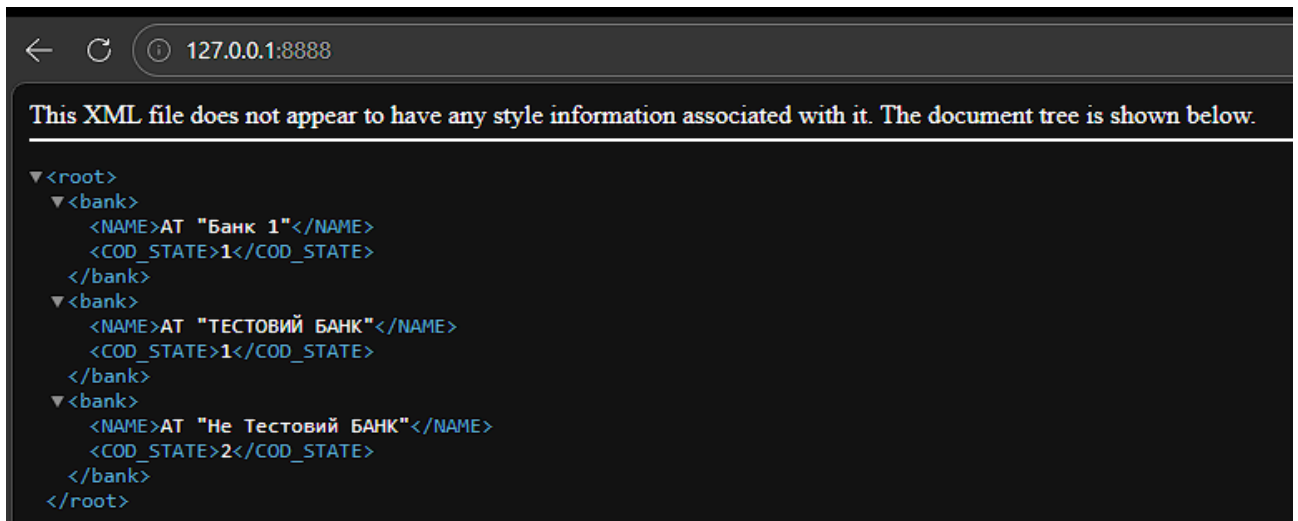
```

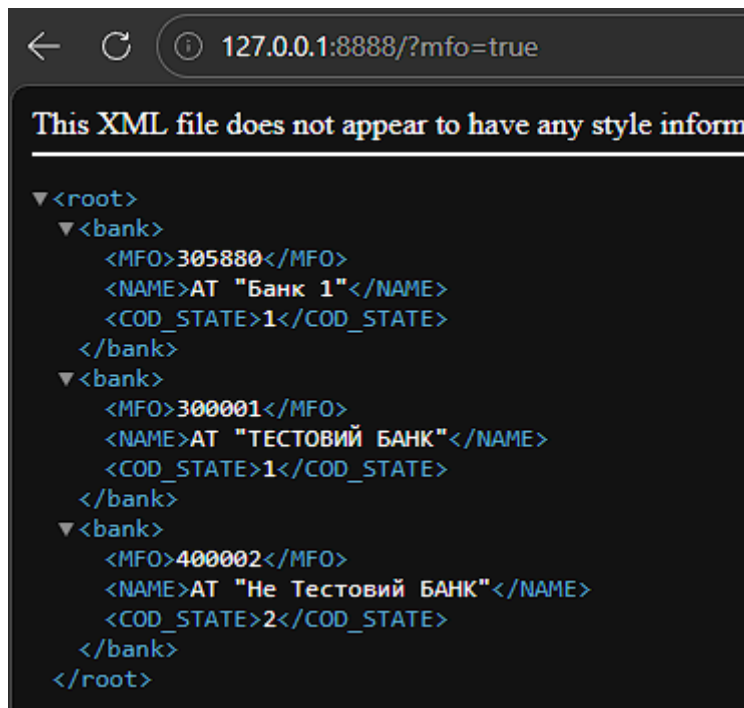
Результат:

```

PS C:\Users\boec7\Desktop\Labs2\WEB\backend-course-2025-4> node main.js -i bank_managers.json -h 127.0.0.1 -p 8888
Server is listening on http://127.0.0.1:8888

```

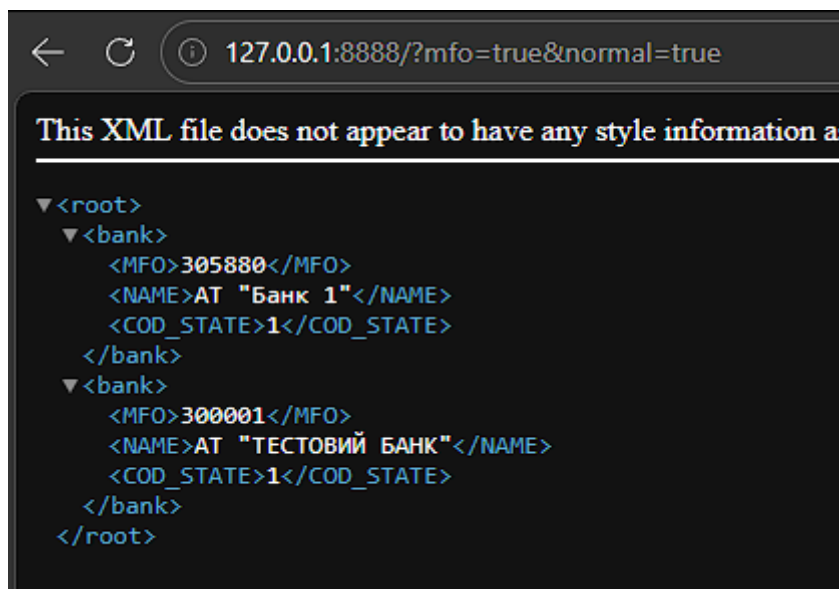




```
← 127.0.0.1:8888/?mfo=true

This XML file does not appear to have any style information associated with it.

<?xml version="1.0"?>
<root>
  <bank>
    <MFO>305880</MFO>
    <NAME>AT "Банк 1"</NAME>
    <COD_STATE>1</COD_STATE>
  </bank>
  <bank>
    <MFO>300001</MFO>
    <NAME>AT "ТЕСТОВИЙ БАНК"</NAME>
    <COD_STATE>1</COD_STATE>
  </bank>
  <bank>
    <MFO>400002</MFO>
    <NAME>AT "Не Тестовий БАНК"</NAME>
    <COD_STATE>2</COD_STATE>
  </bank>
</root>
```



```
← 127.0.0.1:8888/?mfo=true&normal=true

This XML file does not appear to have any style information associated with it.

<?xml version="1.0"?>
<root>
  <bank>
    <MFO>305880</MFO>
    <NAME>AT "Банк 1"</NAME>
    <COD_STATE>1</COD_STATE>
  </bank>
  <bank>
    <MFO>300001</MFO>
    <NAME>AT "ТЕСТОВИЙ БАНК"</NAME>
    <COD_STATE>1</COD_STATE>
  </bank>
</root>
```

Висновок: під час виконання лабораторної роботи я навчився робити асинхронні виклики для читання файлу, обробляти запити та надсилати відповідь з допомогою вбудованого модуля HTTP